TenderSense AI — System architecture

Explainable tender evaluation: OCR + criteria extraction → rule engine + optional LLM → officer UI, review queue, and hash-chained audit trail.

`MIT · monorepo · 3 services`

EXPERIENCE

Frontend — Vite · React 18 · TypeScript

- Dashboard, tender workspace, **reasoning graph** (SVG)
- Review queue, audit log, i18n (EN / हिं)
- JWT access + **HttpOnly refresh** cookie to API

CONTROL PLANE

Backend — Go · Gin · PostgreSQL

- Auth (rotation, JTI denylist, `access_token_version`), RBAC, rate limits
- Tenders / bidders / documents / criteria CRUD
- Async **evaluation_jobs** + worker → AI service
- Optional **MinIO/S3** presigned URLs, security headers

HTTPS + Bearer · CORS via `ALLOWED_ORIGINS` → **REST** `/api/v1/*` → **SQL** + migrations

DATA & CACHE

PostgreSQL · Redis (optional)

- Tenders, criteria, decisions, documents, users, refresh tokens
- Redis: eval rate limits, JTI cache, session role/TV cache
- Shared `DATA_DIR` with AI for uploads (or S3 mirror)

INTELLIGENCE

AI service — FastAPI · Python 3.12

- **OCR** pipeline (pdfplumber / Tesseract / optional Paddle)
- **/v1/extract-criteria**, **/v1/evaluate** decision engine
- Optional **Anthropic / Groq**; sovereign `LLM_BACKEND=disabled`

SECURITY & TRUST (SUMMARY)

Short-lived access JWTs · rotating refresh (DB + HttpOnly cookie) · JTI revocation + cleanup · password reset / logout-all bumps token version · Redis-backed limits · paginated decisions with `X-Total-Count` — see `docs/SECURITY.md`.

DEPLOYMENT SKETCH

Vercel / static CDN → API (Fly or Render) → managed Postgres + Redis → separate **AI_SERVICE_URL** worker · optional object store for document mirror and presign.